

2

Decentralization

Decentralization is not a new concept; it has been used in strategy, management, and governance for a long time. The basic idea of decentralization is to distribute control and authority to peripheries instead of one central authority being in full control of the organization. This results in several benefits for organizations. such as increased efficiency, quicker decision making, better motivation, and a reduced burden on top management.

In this chapter, the concept of decentralization will be discussed in the context of blockchain; the goals of both are similar, whereby no single central authority is in control. Methods of decentralization and routes to decentralization with some examples will also be presented. Also, the decentralization of the blockchain ecosystem, decentralized applications, and platforms for decentralization will be discussed in detail. Many exciting applications and ideas emerge out of the decentralized blockchain technology, and will be introduced in this chapter.

Decentralization using blockchain

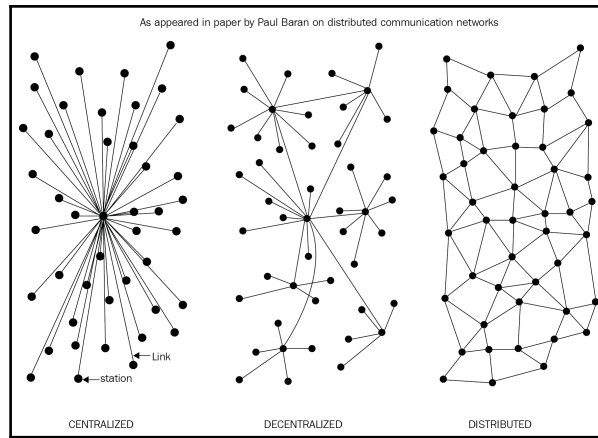
Decentralization is a core benefit and service provided by the blockchain technology. Blockchain by design is a perfect vehicle for providing a platform that does not need any intermediaries and can function with many different leaders chosen via consensus mechanisms. This model allows anyone to compete to become the decision-making authority. This competition is governed by a consensus mechanism and the most commonly used method is known as **Proof of Work (PoW)**.

Decentralization is applied in varying degrees from semi-decentralized to fully decentralized depending on the requirements and circumstances. Decentralization can be viewed from a blockchain perspective as a mechanism that provides a way to remodel existing applications and paradigms or build new applications in order to give full control to users.

Information and communication technology (ICT) has conventionally been based on a centralized paradigm whereby database or application servers are under the control of a central authority, such as a system administrator. With bitcoin and the advent of the blockchain technology, this model has changed and now the technology that allows anyone to start a decentralized system (and operate it with no single point of failure or single trusted authority) is available. It can either be run autonomously or by requiring some human intervention depending on the type and model of governance used in the decentralized application running on the blockchain.

An upcoming diagram shows different types of system that currently exist, that is, central, distributed, and decentralized. This concept was first published in 1964 in a paper by *Paul Baran* on *distributed communication networks* in the context of communication networks.

Centralized systems are conventional (client–server) IT systems whereby there is a single authority that controls the system and is solely in-charge of all operations on the system. All users of a central system are dependent on a single source of service. Online service providers, such as eBay, Google, Amazon, Apple's App Store, and the majority of other providers, use this common model of delivering services. On the other hand, in a distributed system, the data and computation are spread across multiple nodes in the network. Sometimes, this term is confused with parallel computing. While there is an overlap in the definition, the main difference between both these systems is that in a parallel system, computation is performed by all nodes simultaneously in order to achieve a result, whereas in a distributed system, computation may not happen in parallel and data is only replicated on multiple nodes that users view as a single coherent system. Both of these models are used with variations in order to achieve failure tolerance and speed. In this model, there is still a central authority that has control over all nodes and governs processing. This means that the system is still centralized in nature.



Different types of network/system

The key difference between a decentralized system and distributed system is that in a distributed system, there still exists a central authority that governs the entire system, whereas in a decentralized system, no such authority exists. A decentralized system is a type of network whereby nodes are not dependent on a single master node; instead, control is distributed among many nodes. For example, this is analogous to a model where each department in an organization has its own database server that they are in charge of, thus taking away the power from the central server and distributing it to the sub-departments that manage their own databases.

A real innovation in the decentralized paradigm that has started this new era of decentralization applications is decentralized consensus, which was introduced with bitcoin. This enables a user to agree on something via a consensus algorithm without the need for a central trusted third party, intermediary, or service provider.

Methods of decentralization

There are two methods that can be used to achieve decentralization. These methods are discussed in detail in the following sections.

Disintermediation

This can be explained with the help of an example. Imagine you want to send money to your friend in another country. You go to a bank that will transfer your money to the bank in the country of your choice for a fee. In this case, the bank keeps a central database that is updated, confirming that you have sent the money. With blockchain technology, it is possible to send this money directly to your friend without the need for a bank. All you need is the address of your friend on the blockchain. This way, the intermediary is no longer required and decentralization is achieved by disintermediation. However, it is debatable how practical decentralization is in the financial sector by disintermediation due to heavy regulatory and compliance requirements. Nevertheless, this model can be used not only in finance but also in many other different industries.

Through competition

In this method, a group of service providers compete with each other in order to be selected for the provision of services by the system. This paradigm does not achieve complete decentralization, but to a certain degree ensures that an intermediary or service provider is not monopolizing the service. In the context of blockchain technology, a system can be envisioned in which smart contracts can choose an external data provider from a large number of providers based on their reputation, previous score, reviews, and quality of service. This will not result in full decentralization, but it allows smart contracts to make a free choice based on the criteria mentioned earlier. This way, an environment of competition is cultivated among service providers, whereby they compete with each other to become the data provider of choice.

In the following diagram, varying levels of decentralization are shown. On the left-hand side, there is a conventional approach where a central system is in control; on the right-hand side, complete disintermediation is achieved; and in middle, competing intermediaries or service providers are shown. In the middle, intermediaries or service providers are selected based on reputation or voting, thus achieving partial decentralization.

By answering these four questions, it can be shown how a payment system can be decentralized. Based on the preceding answers, it can be stated that the money transfer system can be decentralized by removing the intermediary and will be implemented on the Bitcoin blockchain with security guarantee provided via Atomicity.

Similarly, this framework can be used for any other system that needs to be evaluated for decentralization. By answering these four simple questions, it becomes quite clear as to what approach can be taken to decentralize the system.

Blockchain and full ecosystem decentralization

In order to achieve complete decentralization, it is necessary that the environment around the blockchain is also decentralized. Blockchain itself is a distributed ledger that runs on top of conventional systems. These elements include storage, communication, and computation. There are other factors, such as Identity and Wealth, that are traditionally based on centralized paradigms and there's a need to decentralize these aspects too in order to achieve a fully decentralized ecosystem.

Storage

Data can be stored directly in a blockchain, and with this, it does achieve decentralization, but a major disadvantage of this approach is that blockchain is not suitable for storing large amounts of data by design. It can store simple transactions and some arbitrary data but is certainly not suitable for storing images or large blobs of data, as is the case in traditional database systems. A better alternative is to use **distributed hash tables (DHTs)**. DHTs were originally used in peer-to-peer file sharing software, such as BitTorrent, Napster, Kazaa, and Gnutella. DHT research was made popular by CAN, Chord, Pastry, and Tapestry projects. BitTorrent turns out to be the most scalable and fast network, but the issue is that there is no incentive for users to keep the files indefinitely. Users do not usually keep files permanently, and if nodes leave the network that has data required by someone, there is no way to retrieve it except having the required nodes rejoin the network again so that the files become available once more. Two main requirements here are high availability and link stability, which means that data should be available when required and network links should also always be accessible. **Inter Planetary File System (IPFS)** by Juan Benet possesses both of these properties and the vision is to provide a decentralized World Wide Web by replacing the HTTP protocol. IPFS uses Kademlia DHT and merkle **DAG (Directed Acyclic Graph)** to provide the storage and searching functionality, respectively.

The incentive mechanism is based on a protocol known as Filecoin that pays incentives to nodes that store data using the BitSwap mechanism. The BitSwap mechanism allows nodes to keep a simple ledger of bytes sent or bytes received under a one-to-one relationship. Also, a Git-based version control mechanism is used in IPFS to provide structure and control over the versioning of data.

There are other alternatives, such as Ethereum swarm, storj, and maidsafe. Ethereum has its own decentralized and distributed ecosystem that uses Swarm for storage and the whisper protocol for communication. Maidsafe is aiming to provide a decentralized World Wide Web. All these projects will be discussed later in the book in more detail.

BigChainDB is another storage layer decentralization project aimed at providing a scalable, fast, and linearly scalable decentralized database as opposed to a traditional filesystem. BigChainDB complements decentralized processing platforms and file systems such as Ethereum and IPFS.

Communication

It is generally considered that the Internet (the communication layer in blockchain) is decentralized. This is true to some extent as the original vision of the Internet was to develop a decentralized system. Services such as e-mail and online storage are all now based on a paradigm where the service provider is in control and users trust them to give them access to the service when required. This model is based on the trust of the central authority (the service provider) and users are not in control of their data; even passwords are stored on trusted third-party systems. There is a need to provide control to individual users in such a way that access to their data is guaranteed and is not dependent on a single third party. Access to the Internet (the communication layer) is based on Internet service providers (ISPs) that act as a central hub for Internet users. If the ISP is shut down for political or any other reasons, then no communication is possible in this model. An alternative is to use mesh networks. Even though they are limited in functionality as compared to the Internet, they still provide a decentralized alternative where nodes can talk directly to each other without a central hub such as an ISP.



An example of a Meshnet is Firechat (<http://www.opengarden.com/firechat.html>), which allows iPhone users to communicate with each other directly in a peer-to-peer fashion without the Internet.

Now imagine a network that allows users to be in control of their communication; no one can shut it down for political or censorship reasons. This could be the next step toward decentralizing communication networks in the blockchain ecosystem. It must be noted that this model may only be required in a jurisdiction where the Internet is censored and controlled by the government.

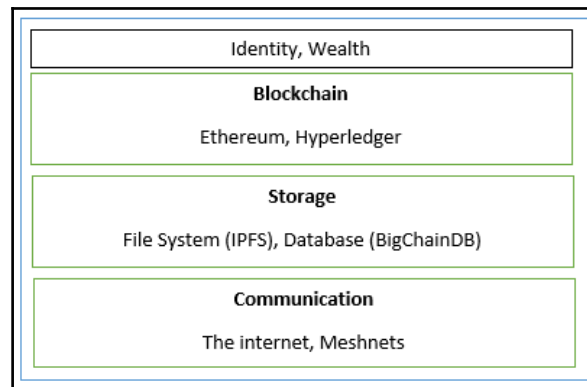
As mentioned earlier, the original vision of the Internet was to build a decentralized network; however, over the years, with the advent of large-scale service providers such as Google, Amazon, and eBay, the control is shifting toward the big players. For example, e-mail is a decentralized system at its core; anyone can run an e-mail server with minimal effort and can start sending and receiving e-mails, but there is a better alternative available that is already providing a managed service for end users, so there is a natural inclination toward selecting a centralized service as it is more convenient and free. Free services, however, are being offered at the cost of valuable personal data and many users are not aware of this fact. This is one example that shows how the Internet has moved toward centralization. Blockchain has once again given this vision of decentralization to the world and now concerted efforts are being made to harness this technology and gain the benefits that it can provide.

Computation

Decentralization of computing or processing is achieved by a blockchain technology such as Ethereum, where smart contracts with embedded business logic can run on the network. Other blockchain technologies also provide similar processing layer platforms where business logic can run over the network in a decentralized manner.

The following diagram shows the decentralized ecosystem overview where, on the bottom layer, Internet or Meshnets provides a decentralized communication layer, then a storage layer uses technologies such as IPFS and BigChainDB to enable decentralization, and finally, you see the blockchain that serves as a decentralized processing layer. Blockchain can, in a limited way, provide a storage layer too, but that seriously hampers the speed and capacity of the system; therefore, other solutions such as IPFS and BigChainDB are more suitable to store large amounts of data in a decentralized way. At the top, the Identity and Wealth layers are shown. Identity on the Internet is a very big topic and systems such as bitAuth and OpenID have provided authentication and identification services with varying degrees of decentralization and security assumptions.

Blockchain is capable of providing solutions to various problems. A concept relevant to Identity known as *Zooko's Triangle* requires that a naming system in a network protocol be secure, decentralized, and meaningful to humans. It is conjectured that a system can have only two of these properties simultaneously, but with the advent of blockchain, in the form of Namecoin, this problem was resolved. This, however, is not a panacea and comes with its own challenges, such as reliance on users to store and maintain private keys securely. This opens up other general questions about the suitability of decentralization. Perhaps decentralization is not appropriate in every scenario. Well-reputed centralized systems tend to work better in many cases.



There are many projects underway that are developing solutions for a wider distributed blockchain system.

With the emergence of the decentralization paradigm, different terminologies and buzz words are now appearing in the media and in academic literature. With the advent of the blockchain technology, it is now possible to build software versions of traditional physical organizations. In the context of decentralization, the upcoming concepts are worth discussing.

Smart contract

A smart contract can be thought of as a small decentralized program. Smart contracts do not necessarily need a blockchain to run; however, due to the security benefits that the blockchain technology provides, it is now becoming almost a standard to use blockchain as a decentralized execution platform for smart contracts. A smart contract usually contains some business logic and a limited amount of data. Actors or participants in the blockchain use these smart contracts or they run autonomously on behalf of the network participants.

These small programs reside on the blockchain and execute business logic if some specific criteria are met. More information on smart contracts will be provided in *Chapter 6, Smart Contracts*, which is dedicated to a detailed discussion of smart contracts.

Decentralized organizations

Decentralized organization (DOs) are software programs that run on a blockchain and are based on the idea of real human organizations with people and protocols. Once a DO, in the form of a smart contract or a set of smart contracts, is added to the blockchain, it becomes decentralized and parties interact with each other based on the code defined within the DO software.

Decentralized autonomous organizations

Just like DOs, a **Decentralized autonomous organization (DAO)** is also a computer program that runs on top of a blockchain and embedded within it are governance and business logic rules. DAO and DO are basically the same thing, but the main difference is that DAOs are autonomous, which means that they are fully automated and contain artificially intelligent logic, whereas DOs lack this feature and rely on human input in order to execute business logic.

Ethereum blockchain led the way with the introduction of DAOs for the first time. In DAO, the code is considered the governing entity rather than humans or paper contracts. A *Curator*, however, is a human entity that participates as someone who maintains this code and acts as a proposal evaluator for the community. DAOs are capable of hiring external *Contractors* if enough input is received from the token holders (participants). The most famous DAO project is *The DAO* (<https://daohub.org>) as it raised 168 million US dollars in its crowd-funding phase. The DAO project was designed to be a venture capital fund which was aimed at providing a decentralized business model with no single entity as an owner. Unfortunately, this was hacked due to a bug in the DAO code and millions of dollars' worth of **Ether currency (ETH)** were siphoned out of the DAO into a child DAO created by the hackers. It required a hard fork on the Ethereum blockchain to reverse the impact of the hack and initiate the recovery of the funds. This incident opened up a debate on the security, quality, and the need for thorough testing of the code in smart contracts in order to ensure integrity and adequate control. There are projects underway, especially in Academia, that are looking to formalize smart contract coding.

Currently, DAOs do not have any legal status even though they may contain some intelligent code that enforces some protocols and conditions, but these rules have no value in the current real-world legal system. One day, perhaps an *autonomous agent* that is commissioned and permissioned by a law enforcement agency or a regulator containing rules and regulations could be embedded in a DAO, to ensure the integrity of the DAO from a legal and compliance perspective. An **Autonomous Agent (AA)** is a piece of code that runs without human intervention. The fact that DAOs are purely decentralized entities makes it possible to run them in any physical jurisdiction. Therefore, they raise a big question as to how a current legal system would work with such a varied mix of different jurisdictions and geographies.

Decentralized autonomous corporations

DAOs, **Decentralized autonomous corporations (DACs)** are a similar concept but are considered a smaller subset of DAOs. The definitions of DACs and DAOs can sometimes overlap, but a general difference is that DAOs are usually considered to be nonprofit, whereas DACs can make money via shares offered to the participants and by paying dividends. These corporations can run a business automatically without human intervention based on the logic programmed within them.

Decentralized autonomous societies

Decentralized autonomous societies (DASs) are a concept whereby entire societies can function on a blockchain with the help of multiple complex smart contracts and a combination of DAOs and **Decentralized applications (DAPPs)** running autonomously. This model does not mean an outlaw approach, nor is it based on a totally libertarian ideology; instead, many services that a government offers can be delivered via blockchain, such as Government Identity Card systems, passport issuance, and records of deeds, marriages, and births. Another theory is that, if a government is corrupt and central systems do not provide the satisfactory levels of trust that a society needs, then the society can start its own virtual society on a blockchain that is driven by decentralized consensus and is transparent. This might be seen as a libertarian or cypherpunk dream but is entirely possible on a blockchain.

Decentralized applications

All ideas mentioned earlier come under the larger umbrella of decentralized applications. All DAOs, DACs, and DOs are basically decentralized applications that run on top of a blockchain in a peer-to-peer network. This is the latest advancement in technology with regard to decentralization. Decentralized applications or DAPPs are software programs that can run on their own blockchain, use another already existing established blockchain, or use only protocols of an existing blockchain solution. These are called Type I, Type II, and Type III DAPPs.

Requirements of a decentralized application

In order for an application to be considered a decentralized application, it must meet the following criteria. This definition was provided by *David Johnston* and others in their whitepaper called *The General Theory of Decentralized Applications, Dapps*:

1. The DAPP should be fully open source and autonomous and no single entity should be in control of a majority of its tokens. All changes to the application must be consensus-driven based on the feedback given by the community.
2. Data and records of operations of the application must be cryptographically secured and stored on a public, decentralized blockchain in order to avoid any central points of failure.
3. A cryptographic token must be used by the application in order to provide access and rewards to those who contribute value to the applications, for example, miners in bitcoin.
4. The tokens must be generated by the decentralized application according to a standard cryptographic algorithm. This generation of tokens acts as a proof of the value to contributors (for example, miners).

Operations of a DAPP

Establishment of consensus by a DAPP can be achieved using consensus algorithms such as Proof of Work and Proof of Stake. So far, only PoW has been found to be incredibly resistant to 51% attacks, as is evident from bitcoin. Furthermore, a DAPP can distribute tokens (coins) via mining, fundraising, and development.

Examples

Examples of some decentralized applications are provided here.

KYC-Chain

This application provides a facility to manage **Know Your Customer (KYC)** data in a secure and convenient way based on smart contracts.

OpenBazaar

This is a decentralized peer-to-peer network that allows commercial activities directly between sellers and buyers instead of relying on a central party, as opposed to conventional providers such as eBay and Amazon. It should be noted that this system is not built on top of a blockchain; instead, distributed hash tables are used in a peer-to-peer network in order to enable direct communication and data sharing between peers. It makes use of bitcoin as a payment network, however.

Lazooz

This is a decentralized equivalent of Uber. It allows peer-to-peer ride sharing and users can be incentivized by *proof of movement* and can earn Zooz coins.



There are many other DAPPS that have been built on the Ethereum blockchain and are showcased at <http://dapps.ethercasts.com/>.

Platforms for decentralization

There are many platforms available for decentralization now. Many companies around the world have introduced platforms that promise to make distributed application development easy, accessible, and secure for users. Some prominent names are discussed here.

Ethereum

Ethereum tops the list as being the first blockchain that introduced a Turing-complete language and the concept of a virtual machine. This is in contrast to the limited scripting language in bitcoin and many other cryptocurrencies. With the availability of this Turing-complete language called Solidity, endless possibilities have opened for the development of decentralized applications. This was proposed in 2013 by *Vitalik Buterin* and provides a public blockchain to develop smart contracts and decentralized applications. Currency tokens on Ethereum are called Ethers.

Maidsafe

Maidsafe provides a **SAFE (Secure Access for Everyone)** network that is made up of unused computing resources, such as storage, processing power, and the data connections of its users. The files on the network are divided into small chunks of data that are encrypted and distributed throughout the network randomly. This data can only be retrieved by its respective owner. One key innovation is that duplicate files are automatically rejected on the network, which helps reduce the need for additional computing resources to manage the load. It uses Safecoin as a token to incentivize its contributors.

Lisk

Lisk is a blockchain application development and cryptocurrency platform. It allows developers to use JavaScript to build decentralized applications and host them in their own respective sidechains. Lisk uses the **Delegated Proof of Stake (DPOS)** mechanism for consensus whereby 101 nodes can be elected to secure the network and propose blocks. It uses the Node.js and JavaScript backend whereas the frontend allows the use of standard technologies, such as CSS3, HTML5, and JavaScript. Lisk uses LSK coin as a currency on the blockchain. Another derivative of Lisk is Rise, which is a Lisk-based decentralized application and digital currency platform. It has more focus on the security of the system.

A more practical introduction to these platforms and others will be supplied in later chapters.

Introduction

Cryptography is the science of making information secure in the presence of adversaries. It provides a means of secure communication in the presence of adversaries with assumed limitless resources. Ciphers are used to encrypt data so that if intercepted by an adversary, the data is meaningless to them without decryption, which requires the secret key.

Cryptography is generally used to provide a confidentiality service. On its own, it cannot be considered a complete solution but serve as a crucial building block within a larger security system to address a security problem.

Cryptography provides various security services, such as **Confidentiality**, **Integrity**, **Authentication**, (Entity Authentication and Data origin authentication) and **non-repudiation**. Additionally, accountability is also required in various security systems.

Before discussing cryptography further, there are some mathematical terms and concepts that need to be explained first in order to fully understand the material provided later in this chapter. The next section introduces these concepts. It should be noted that this section is intended as a basic introduction. An explanation with proofs and relevant background for all these terms will require rather involved mathematics, which is beyond the scope of this book. More details on these topics can be found in any standard number theory, algebra, or cryptography text book.

Mathematics

As the subject of cryptography is based on mathematics, this section will introduce some basic concepts that will help you understand the concepts later in the chapter.

Set

A set is a collection of distinct objects, for example, $X = \{1, 2, 3, 4, 5\}$.

Group

A group is a commutative set with one operation that combines two elements of the set. The group operation is closed and associated with an identity element defined. Additionally, each element in the set has an inverse. Closure (closed) means that if, for example, elements A and B are in the set, then the resultant element after performing operation on the elements is also in the set. Associative means that the grouping of elements does not affect the result of the operation.

Field

A field is a set that contains both additive and multiplicative groups. More precisely, all elements in the set form an additive and multiplicative group. It satisfies specific axioms for addition and multiplication. For all group operations, the distributive law is also applied. The law dictates that the same sum or product will be produced even if any terms or factors are reordered.

A finite field

A finite field is a field with a finite set of elements. Also known as Galois fields, these structures are of particular importance in cryptography as they can be used to produce accurate and error-free results of arithmetic operations. For example, prime finite fields are used in elliptic curve cryptography to construct discrete logarithm problem.

Order

This is the number of elements in a field. It is also known as the cardinality of the field.

Prime fields

This is a finite field with a prime number of elements. It has specific rules for addition and multiplication, and each nonzero element in the field has an inverse. Addition and multiplication operations are performed modulo p .

Ring

If more than one operation can be defined over an abelian group, that group becomes a ring. There are also certain properties that need to be satisfied. A ring must have closure and associative and distributive properties.

A cyclic group

A cyclic group is a type of group that can be generated by a single element called the group generator. In other words, if the group operation is repeatedly applied to a particular element in the group, then all elements in the group can be generated.

An abelian group

An abelian group is formed when the operation on the elements of a set is commutative. Commutative law basically means that changing the order of the elements does not affect the result of the operation, for example, $A \times B = B \times A$.

Modular arithmetic

Also known as clock arithmetic, numbers in modular arithmetic wrap around when they reach a certain fixed number. This fixed number is a positive number called modulus and all operations are performed with regard to this fixed number. In an analogy to a clock, there are number from 1 to 12. When it reaches 12, the number 1 starts again. In other words, this arithmetic deals with the remainders after the division operation. For example, $50 \bmod 11$ is 6 because $50 / 11$ leaves a remainder of 6.

This completes a basic introduction to some mathematical concepts; in the next section, you will be introduced to cryptography.

Cryptography

As discussed earlier, cryptography provides various security services, and these security services are discussed here.

Confidentiality

Confidentiality is the assurance that information is only available to authorized entities.

Integrity

Integrity is the assurance that information is modifiable only by authorized entities.

Authentication

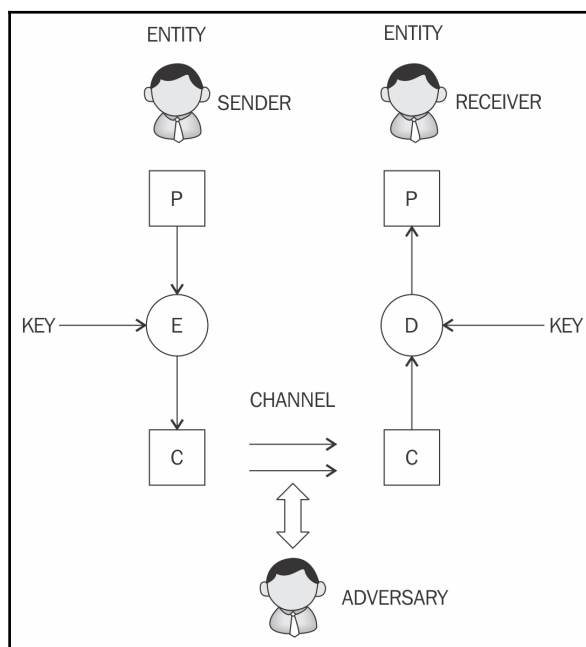
Authentication provides assurance about the identity of an entity or the validity of a message. There are two types of authentications, discussed here.

Cryptographic primitives

Cryptographic primitives are the basic building blocks of a security protocol or system. In the following section, you are introduced to cryptographic algorithms that are essential for the building of secure protocols and systems. A **security protocol** is a set of steps taken in order to achieve required security goals by utilizing appropriate security mechanisms.

Various types of security protocols are in use, such as **authentication protocols**, **non-repudiation protocols**, and **key management protocols**.

A generic cryptography model is shown in the following diagram:



A model showing the generic encryption and decryption model

In the preceding diagram, **P**, **E**, **C**, and **D** represents Plain text, Encryption, Cipher text, and Decryption, respectively. Also, based on the model shown earlier, it is worth explaining various concepts such as entity, sender, receiver, adversary, key, and a channel.

- **Entity:** It is either a person or a system that sends, receives, or performs operations on data
- **Sender:** Sender is an entity that transmits the data
- **Receiver:** Receiver is an entity that takes delivery of the data

- **Adversary:** This is an entity that tries to circumvent the security service
- **Key:** A key is some data that is used to encrypt or decrypt data
- **Channel:** Channel provides a medium of communication between entities

Cryptography is mainly divided into two categories, namely symmetric and asymmetric cryptography.

Symmetric cryptography

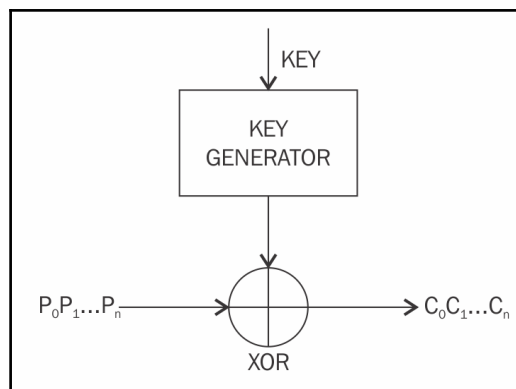
Symmetric cryptography refers to a type of cryptography whereby the key that is used to encrypt the data is the same for decrypting the data, and thus it is also known as a shared key cryptography. The key must be established or agreed on before the data exchange between the communicating parties. This is the reason it is also called **secret key cryptography**.

There are two types of symmetric ciphers, stream ciphers and block ciphers. **Data Encryption Standard (DES)** and **Advanced Encryption Standard (AES)** are common examples of block ciphers, whereas RC4 and A5 are commonly used stream ciphers.

Stream ciphers

These ciphers are encryption algorithms that apply encryption algorithms on a bit-by-bit basis to plain text using a key stream. There are two types of stream ciphers: synchronous and asynchronous. Synchronous stream ciphers are ones where key stream is dependent only on the key, whereas asynchronous stream ciphers have a key stream that is also dependent on the encrypted data.

In stream ciphers, encryption and decryption are basically the same function because they are simple modulo 2 additions or XOR operation. The key requirement in stream ciphers is the security and randomness of key streams. Various techniques have been developed to generate random numbers, and it's vital that all key generators be cryptographically secure:



Operation of a stream cipher

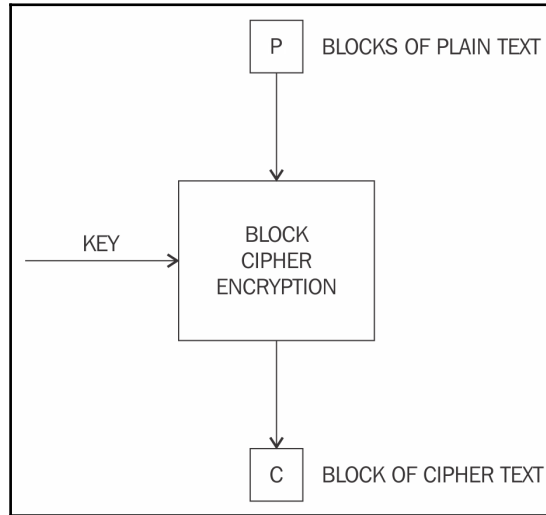
Block ciphers

These are encryption algorithms that break up a text to be encrypted (plain text) into blocks of fixed length and apply encryption block by block. Block ciphers are usually built using a design strategy known as Feistel cipher. Recent block ciphers, such as AES (Rijndael) have been built using a combination of substitution and permutation called **substitution-permutation network (SPN)**.

Feistel ciphers are based on the Feistel network, which is a structure developed by *Horst Feistel*. This structure is based on the idea of combining multiple rounds of repeated operations to achieve desirable cryptographic properties known as confusion and diffusion. Feistel networks operate by dividing data into two blocks (left and right) and process these blocks via keyed round functions.

Confusion makes the relationship between the encrypted text and plaintext complex. This is achieved by substitution in practice. For example, 'A' in plain text is replaced by 'X' in encrypted text. In modern cryptographic algorithms, substitution is performed using lookup tables called S-boxes. The diffusion property spreads the plain text statistically over the encrypted data, which ensures that even if a single bit is changed in the input text, it results in changing at least half (on average) of the bits in the cipher text. Confusion is required to make finding the encryption key very difficult even if many encrypted and decrypted data pairs are created using the same key. In practice, this is achieved by transposition or permutation.

A key advantage of using Fiestel cipher is that encryption and decryption operations are almost identical and only require a reversal of the encryption process in order to achieve decryption. DES is a prime example of Fiestel-based ciphers:



Simplified operation of a block cipher

Various modes of operation for block ciphers are **Electronic Code Book (ECB)**, **Cipher block chaining (CBC)**, **Output Feedback Mode (OFB)**, or **Counter mode (CTR)**. These modes are used to specify the way in which an encryption function would be applied to the plain text. These modes will be explained later in this section, but the first four categories of block cipher encryption modes are introduced here.

Block encryption mode

In this mode, plaintext is divided into blocks of fixed length depending on the type of cipher used and then the encryption function is applied on each block.

Keystream generation modes

In this mode, the encryption function generates a keystream that is then XORed with the plaintext stream in order to achieve encryption.

Message authentication modes

In this mode, a message authentication code is computed as a result of an encryption function. MAC is basically a cryptographic checksum that provides an integrity service. The most common method to generate MAC using block ciphers is CBC-MAC, where some part of the last block of the chain is used as a MAC.

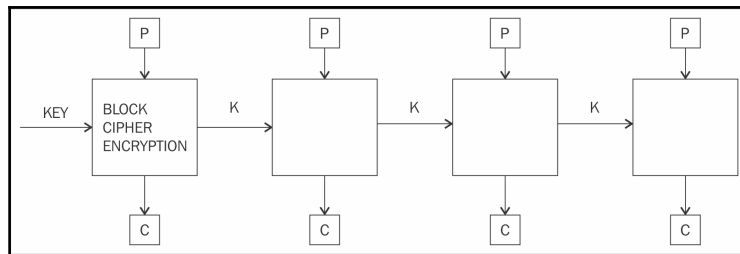
Cryptographic hashes

Hash functions are basically used to compress a message to a fixed length digest. In this mode, block ciphers are used as a compression function to produce a hash of plain text.

The most common block encryption modes are discussed briefly.

Electronic code book

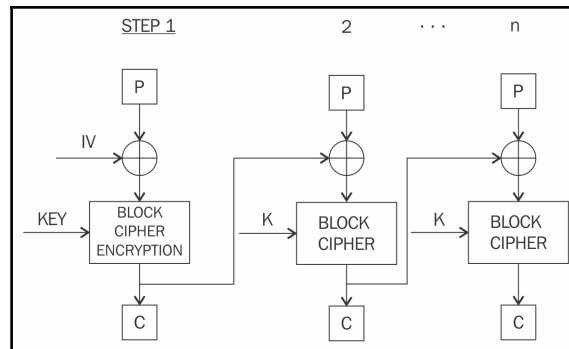
This is a basic mode of operation in which the encrypted data is produced as a result of applying the encryption algorithm one by one separately to each block of plain text. This is the simplest mode but should not be used in practice as it is insecure and can reveal information:



Electronic code book mode for block ciphers

Cipher block chaining

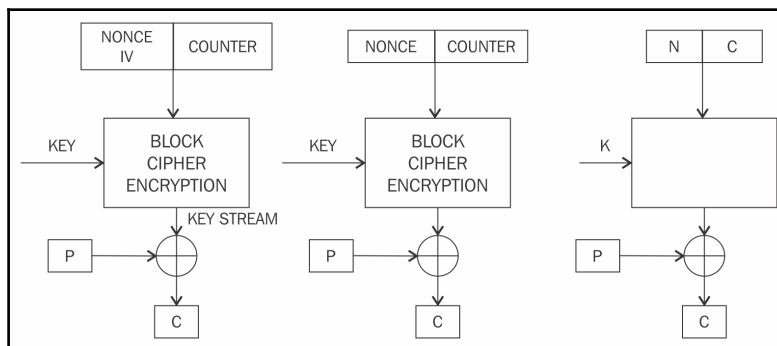
In this mode, each block of plain text is XORed with the previous encrypted block. The CBC mode uses initialization vector IV to encrypt the first block. It is recommended that IV be randomly chosen:



Cipher block chaining mode

Counter mode

The CTR mode effectively uses a block cipher as a stream cipher. In this case, a unique nonce is supplied that is concatenated with the counter value in order to produce a **key stream**:



Counter mode

There are other modes, such as **Cipher Feedback mode (CFB)**, **Galois Counter mode (GCM)**, and **Output Feedback mode**, which are also used in various scenarios.

In the following section, you will be introduced to the design and mechanism of a currently dominant block cipher known as AES. First, some history will be presented with regard to Data Encryption Standard (DES) that led to the development of a new AES standard.

Data Encryption Standard (DES)

DES was introduced by the US **National Institute of Standards and Technology (NIST)** as a standard algorithm for encryption and was in main use during 1980s and 1990s, but it has been not proven to be very resistant against brute force attacks, due to advances in technology and cryptography research. Especially in July 1998, **Electronic Frontier Foundation (EFF)** broke DES using a special purpose machine. DES uses a key of only 56 bits, which has raised some concerns. This problem was addressed with the introduction of **Triple DES (3DES)**, which proposed the usage of a 168-bit key using three 56-bit keys and the same number of executions of the DES algorithm, thus making brute force attacks almost impossible. But other limitations, such as slow performance and 64-bit block size, are not desirable.

Advanced Encryption Standard (AES)

In 2001, after an open competition, an encryption algorithm named Rijndael that was invented by cryptographers *Joan Daemen* and *Vincent Rijmen* was standardized as AES with minor modifications by NIST in 2001. So far, no attack has been found against AES that is better than the brute force method. Original Rijndael allows different key and block sizes of 128-bit, 192-bit, and 256-bits, but in the AES standard, only a 128-bit block size is allowed. However, key sizes of 128-bit, 192-bit, and 256-bit are allowed.

AES steps

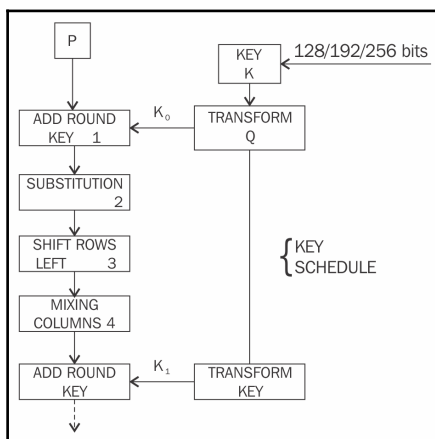
During the AES Algorithm processing, a 4 by 4 array of bytes known as *state* is modified using multiple rounds. Full encryption requires 10 to 14 rounds depending on the size of the key. The following table shows the key sizes and the required number of rounds:

Key size	Number of rounds required
128-bit	10 rounds
192-bit	12 rounds
256-bit	14 rounds

Once the state is initialized with the input to the cipher, four operations are performed in four stages in order to encrypt the input. These stages are AddRoundKey, SubBytes, ShiftRows, and MixColumns:

1. In the AddRoundKey step, the state array is XORed with a subkey, which is derived from the master key.
2. This is the substitution step where a lookup table (S-box) is used to replace all bytes of the state array.
3. This step is used to shift each row except the first one in the state array to the left in a cyclic and incremental manner.
4. Finally, all bytes are mixed in this step in a linear fashion column-wise.

The preceding steps describe one round of AES. In the final round (either 10, 12, or 14 depending on the key size), stage 4 is replaced with Addroundkey to ensure that the first three steps cannot be simply inverted back:



AES block diagram, showing 1st round, in last round mixing step is not performed

Various cryptocurrency wallets use AES encryption to encrypt locally stored data. Especially in bitcoin wallet, AES 256 in the CBC mode is used.

An OpenSSL example of how to encrypt and decrypt using AES

```
:~/Crypt$ openssl enc -aes-256-cbc -in message.txt -out message.bin
enter aes-256-cbc encryption password:
Verifying - enter aes-256-cbc encryption password:
:~/Crypt$ ls -ltr
total 12
```